

OPERATING SYSTEMS AND SYSTEM ENGINEERING

WEEK-1

TASK-1

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Aim:

To develop a C program that demonstrates how the Linux Operating System executes a command entered by the user by creating a child process using `fork()`, executing the command using `exec()`, allowing the parent process to wait using `wait()`, and displaying the Process IDs (PIDs) of both parent and child processes

Software Requirements:

- Ubuntu 24.04 LTS (WSL2)
- GCC Compiler
- Linux Terminal

Procedure:

1. Open the Ubuntu terminal.
2. Create a C source file named `process.c`.
3. Write the program using `fork()`, `execlp()` and `wait()`.
4. Compile the program using the GCC compiler.
5. Execute the program.

6. Enter a Linux command such as pwd, ls, whoami, or date.
7. Observe the Parent Process ID and Child Process ID displayed on the terminal.
8. Verify that the parent process waits until the child process completes execution.

C Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    printf("Enter a Linux command: ");
    scanf("%s", command);
    int pid = fork();
    if (pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }
    if (pid == 0)
    {
        printf("\n----- Child Process ----- \n");
        printf("Child PID : %d\n", getpid());
        execlp(command, command, NULL);
        printf("Invalid Command!\n");
        exit(1);
    }
    else
    {

```

```

    printf("\n----- Parent Process -----\\n");
    printf("Parent PID : %d\\n", getpid());
    printf("Child PID : %d\\n", pid);
    wait(NULL);
    printf("\\nChild Process Completed Successfully.\\n");
}
return 0;
}

```

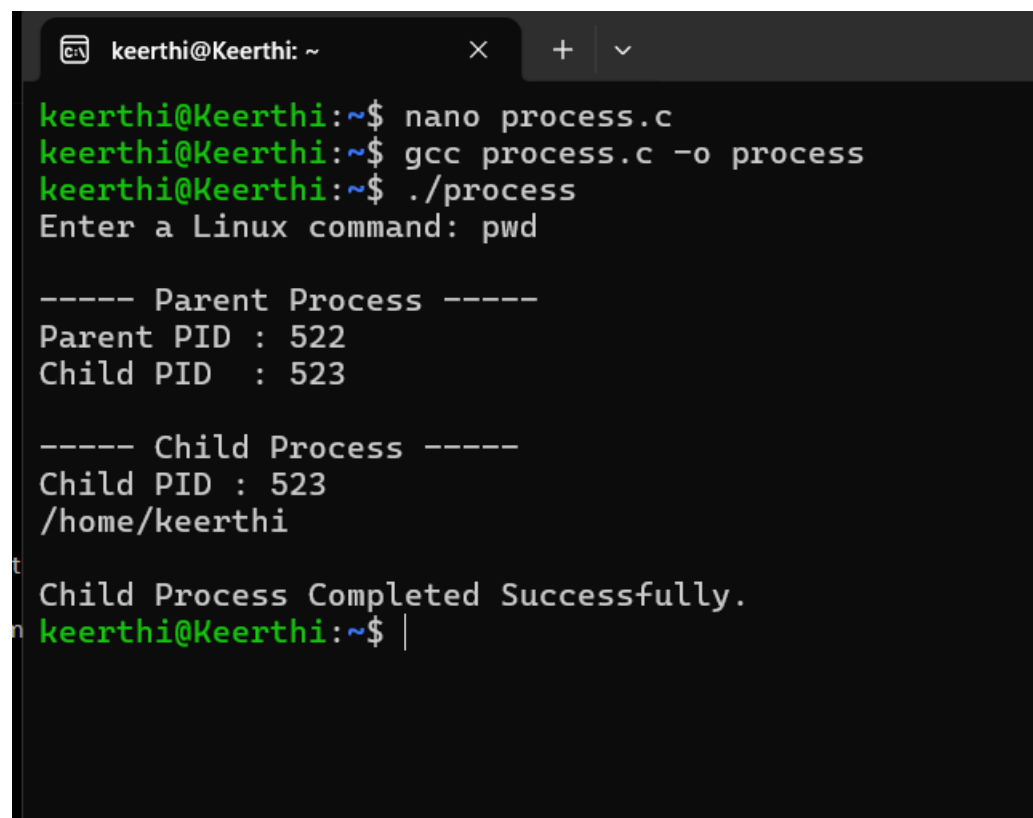
For compilation:

```
gcc process.c -o process
```

Execution

```
./process
```

Output Screenshot



```

keerthi@Keerthi: ~
keerthi@Keerthi:~$ nano process.c
keerthi@Keerthi:~$ gcc process.c -o process
keerthi@Keerthi:~$ ./process
Enter a Linux command: pwd

----- Parent Process -----
Parent PID : 522
Child PID : 523

----- Child Process -----
Child PID : 523
/home/keerthi

Child Process Completed Successfully.
keerthi@Keerthi:~$ |

```

Conclusion

The program successfully demonstrates process creation using `fork()`, command execution using `execlp()`, synchronization using `wait()`, and displays the Process IDs of both parent and child processes in the Linux operating system.